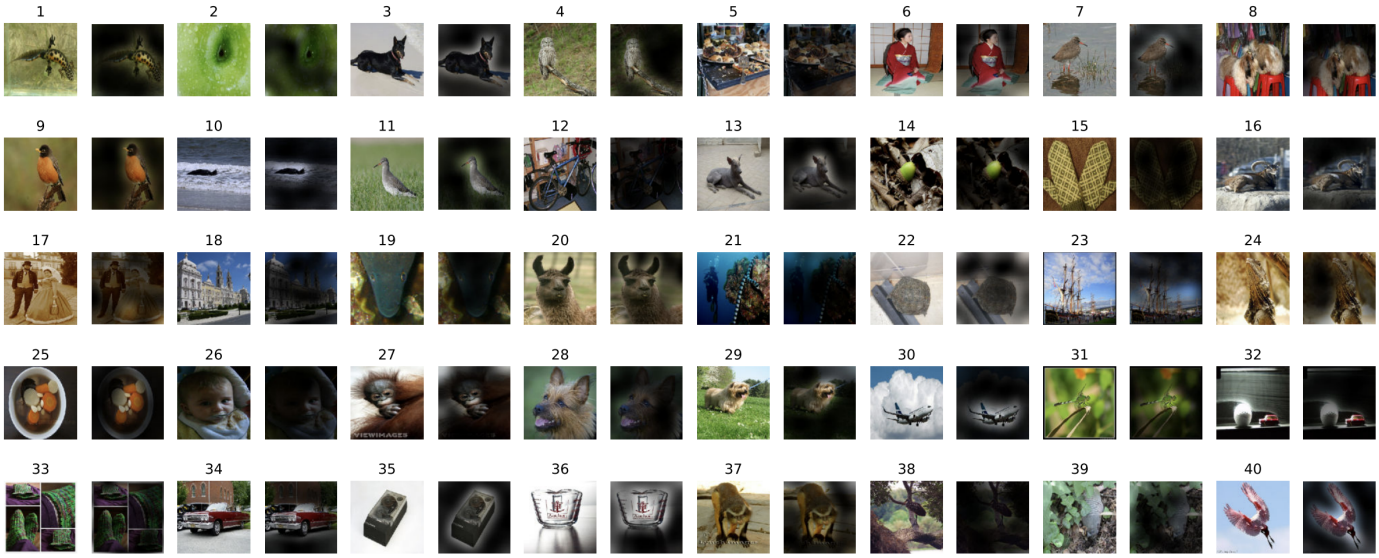


Fig. 9. Attention map results from [34]. The network focuses on the discriminative regions of each image. Figure from [34].

whereupon Eq. 55 can be simplified to:

$$z_i = \sum_{j \in \Omega(i)} F_{\Delta_{ij}}(x_i) x_j + \sum_{j \in \Omega(i)} F_{\Delta_{ij}}(x_j) x_j. \tag{57}$$

The first term can be viewed as collecting information at position i while the second term distributes information at position j. Functions $F_{\Delta_{ij}}(x_i)$ and $F_{\Delta_{ij}}(x_j)$ can be seen as adaptive attention weights.

The above process aggregates global information while emphasizing the relevant features. It can be added to the end of a convolutional neural network as an effective complement to greatly improve semantic segmentation.

3.4 Temporal Attention

Temporal attention can be seen as a dynamic time selection mechanism determining *when to pay attention*, and is thus usually used for video processing. Previous works [171], [172] often emphasise how to capture both short-term and long-term cross-frame feature dependencies. Here, we first summarize representative temporal attention mechanisms and specify process $g(x)$ and $f(g(x), x)$ described as Eq. 1 in Tab. 5, and then discuss various such mechanisms according to the order in Fig. 4.

3.4.1 Self-attention and variants

RNN and temporal pooling or weight learning have been widely used in work on video representation learning to capture interaction between frames, but these methods have limitations in terms of either efficiency or temporal relation modeling.

To overcome them, Li et al. [171] proposed a *global-local temporal representation* (GLTR) to exploit multi-scale temporal cues in a video sequence. GLTR consists of a *dilated temporal pyramid* (DTP) for local temporal context learning and a *temporal self attention* module for capturing global temporal interaction. DTP adopts dilated convolution with dilatation rates increasing progressively to cover various temporal ranges, and then concatenates the various outputs to aggregate multi-scale information. Given input frame-wise features $F = \{f_1, \dots f_T\}$, DTP can be written as:

$$\{f_1^{(r)}, \dots, f_T^{(r)}\} = \text{DConv}^{(r)}(F) \tag{58}$$

$$f'_t = [f_t^{(1)}; \dots f_t^{(2^{n-1})} \dots ; f_t^{(2^{N-1})}] \tag{59}$$

where $\text{DConv}^{(r)}(\cdot)$ denotes dilated convolution with dilation rate r. The self-attention mechanism adopts convolution layers followed by batch normalization and ReLU activation to generate the query $Q \in \mathbb{R}^{d \times T}$, the key $K \in \mathbb{R}^{d \times T}$ and the value $V \in \mathbb{R}^{d \times T}$ based on the input feature map $F' = \{f'_1, \dots f'_T\}$, which can be written as

$$F_{\text{out}} = g(V \text{Softmax}(Q^T K)) + F' \tag{60}$$

where g denotes a linear mapping implemented by a convolution.

The short-term temporal contextual information from neighboring frames helps to distinguish visually similar regions while the long-term temporal information serves to overcome occlusions and noise. GLTR combines the advantages of both modules, enhancing representation capability and suppressing noise. It can be incorporated into any state-of-the-art CNN backbone to learn a global descriptor for a whole video. However, the self-attention mechanism has quadratic time complexity, limiting its application.

3.4.2 TAM

To capture complex temporal relationships both efficiently and flexibly, Liu et al. [172] proposed a *temporal adaptive module* (TAM). It adopts an adaptive kernel instead of self-attention to capture global contextual information, with lower time complexity than GLTR [171].

TAM has two branches, a local branch and a global branch. Given the input feature map $X \in \mathbb{R}^{C \times T \times H \times W}$, global spatial average pooling GAP is first applied to the feature map to ensure TAM has a low computational cost. Then the local branch in TAM employs several 1D convolutions with ReLU

TABLE 5
Representative temporal attention mechanisms sorted by date. ReID = re-identification, Action = action recognition. Ranges means the ranges of attention map. S or H means soft or hard attention. $g(x)$ and $f(g(x), x)$ are the attention process described by Eq. 1. (A) aggregate information via attention map. (I) exploit multi-scale short-term temporal contextual information (II)capture long-term temporal feature dependencies (III) capture local temporal contexts

Category	Method	Publication	Tasks	$g(x)$	$f(g(x), x)$	Ranges	S or H	Goals
Self-attention based methods	GLTR [171]	ICCV2019	ReID	dilated 1D Convs -> self-attention in temporal dimension	(A)	(0,1)	S	(I), (II).
Combine local attention and global attention	TAM [172]	Arxiv2020	Action	a)local: global spatial average pooling -> 1D Convs, b) global: global spatial average pooling -> MLP -> adaptive convolution	(A)	(0,1)	S	(II), (III).

nonlinearity across the temporal domain to produce location-sensitive importance maps for enhancing frame-wise features. The local branch can be written as

$$s = \sigma(\text{Conv1D}(\delta(\text{Conv1D}(\text{GAP}(X))))) \tag{61}$$

$$X^1 = sX. \tag{62}$$

Unlike the local branch, the global branch is location invariant and focuses on generating a channel-wise adaptive kernel based on global temporal information in each channel. For the c-th channel, the kernel can be written as

$$\Theta_c = \text{Softmax}(\text{FC}_2(\delta(\text{FC}_1(\text{GAP}(X)_c)))) \tag{63}$$

where $\Theta_c \in \mathbb{R}^K$ and K is the adaptive kernel size. Finally, TAM convolves the adaptive kernel Θ with X^1_{out}:

$$Y = \Theta \otimes X^1 \tag{64}$$

With the help of the local branch and global branch, TAM can capture the complex temporal structures in video and enhance per-frame features at low computational cost. Due to its flexibility and lightweight design, TAM can be added to any existing 2D CNNs.

3.5 Branch Attention

Branch attention can be seen as a dynamic branch selection mechanism: *which to pay attention to*, used with a multi-branch structure. We first summarize representative branch attention mechanisms and specify process $g(x)$ and $f(g(x), x)$ described as Eq. 1 in Tab. 6, then discuss various ones in detail.

3.5.1 Highway networks

Inspired by the *long short term memory* network, Srivastava et al. [113] proposed *highway networks* that employ adaptive gating mechanisms to enable information flows across layers to address the problem of training very deep networks.

Supposing a plain neural network consists of L layers, and $H_l(X)$ denotes a non-linear transformation on the l-th layer, a highway network can be expressed as

$$Y_l = H_l(X_l)T_l(X_l) + X_l(1 - T_l(X_l)) \tag{65}$$

$$T_l(X) = \sigma(W_l^T X + b_l) \tag{66}$$

where $T_l(X)$ denotes the transform gate regulating the information flow for the l-th layer. X_l and Y_l are the inputs and outputs of the l-th layer.

The gating mechanism and skip-connection structure make it possible to directly train very deep highway networks using simple gradient descent methods. Unlike fixed skip-connections, the gating mechanism adapts to the input, which helps to route information across layers. A highway network can be incorporated in any CNN.

3.5.2 SKNet

Research in the neuroscience community suggests that visual cortical neurons adaptively adjust the sizes of their receptive fields (RFs) according to the input stimulus [174]. This inspired Li et al. [114] to propose an automatic selection operation called *selective kernel* (SK) convolution.

SK convolution is implemented using three operations: split, fuse and select. During split, transformations with different kernel sizes are applied to the feature map to obtain different sized RFs. Information from all branches is then fused together via element-wise summation to compute the gate vector. This is used to control information flows from the multiple branches. Finally, the output feature map is obtained by aggregating feature maps for all branches, guided by the gate vector. This can be expressed as:

$$U_k = F_k(X) \qquad k = 1, \ldots, K \tag{67}$$

$$U = \sum_{k=1}^{K} U_k \tag{68}$$

$$z = \delta(\text{BN}(W\text{GAP}(U))) \tag{69}$$

$$s_k^{(c)} = \frac{e^{W_k^{(c)} z}}{\sum_{k=1}^{K} e^{W_k^{(c)} z}} \quad k = 1, \ldots, K, \quad c = 1, \ldots, C \tag{70}$$

$$Y = \sum_{k=1}^{K} s_k U_k \tag{71}$$

Here, each transformation F_k has a unique kernel size to provide different scales of information for each branch. For efficiency, F_k is implemented by grouped or depthwise convolutions followed by dilated convolution, batch normalization and ReLU activation in sequence. $t^{(c)}$ denotes the c-th element of vector t, or the c-th row of matrix t.

SK convolutions enable the network to adaptively adjust neurons' RF sizes according to the input, giving a notable improvement in results at little computational cost. The gate mechanism in SK convolutions is used to fuse information from multiple branches. Due to its lightweight design, SK

TABLE 6
Representative branch attention mechanisms sorted by date. Cls = classification, Det=Object Detection. $g(x)$ and $f(g(x), x)$ are the attention process described by Eq. 1.Ranges means the ranges of attention map. S or H means soft or hard attention. (A) element-wise product. (B) channel-wise product. (C) aggergate information via attention. (I)overcome the problem of vanishing gradient (II) dynamically fuse different branches. (III) adaptively select a suitable receptive field (IV) improve the performance of standard convolution (be) dynamically fuse different convolution kernels.

Category	Method	Publication	Tasks	$g(x)$	$f(g(x), x)$	Ranges	S or H	Goals
Combine different branches	Highway Network [113]	ICML2015W	Cls	linear layer -> sigmoid	(A)	(0,1)	S	(I), (II).
	SKNet [114]	CVPR2019	Cls	global average pooling -> MLP -> softmax	(B)	(0,1)	S	(II), (III)
Combine different convolution kernels	CondConv [173]	NeurIPS2019	Cls, Det	global average pooling -> linear layer -> sigmoid	(C)	(0,1)	S	(IV), (V).

convolution can be applied to any CNN backbone by replacing all large kernel convolutions. ResNeSt [115] also adopts this attention mechanism to improve the CNN backbone in a more general way, giving excellent results on ResNet [145] and ResNeXt [175].

3.5.3 CondConv

A basic assumption in CNNs is that all convolution kernels are the same. Given this, the typical way to enhance the representational power of a network is to increase its depth or width, which introduces significant extra computational cost. In order to more efficiently increase the capacity of convolutional neural networks, Yang et al. [173] proposed a novel multi-branch operator called CondConv.

An ordinary convolution can be written

$$Y = W * X \tag{72}$$

where $*$ denotes convolution. The learnable parameter W is the same for all samples. CondConv adaptively combines multiple convolution kernels and can be written as:

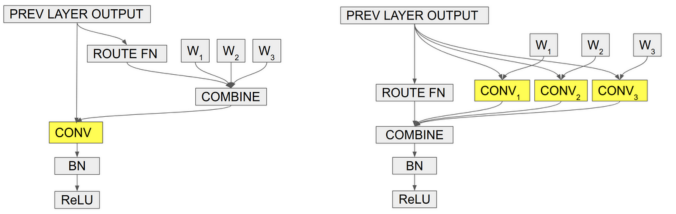
$$Y = (\alpha_1 W_1 + \cdots + \alpha_n W_n) * X \tag{73}$$

Here, α is a learnable weight vector computed by

$$\alpha = \sigma(W_r(\mathrm{GAP}(X))) \tag{74}$$

This process is equivalent to an ensemble of multiple experts, as shown in Fig. 10.

CondConv makes full use of the advantages of the multi-branch structure using a branch attention method with little computing cost. It presents a novel manner to efficiently increase the capability of networks.



(a) CondConv: $(\alpha_1 W_1 + \ldots + \alpha_n W_n) * x$ (b) Mixture of Experts: $\alpha_1(W_1 * x) + \ldots + \alpha_n(W_n * x)$

Fig. 10. CondConv [173]. (a) CondConv first combines different convolution kernels and then uses the combined kernel for convolution. (b) Mixture of experts first uses multiple convolution kernels for convolution and then merges the results. While (a) and (b) are equivalent, (a) has much lower computational cost. Figure is taken from [173].

3.5.4 Dynamic Convolution

The extremely low computational cost of lightweight CNNs constrains the depth and width of the networks, further decreasing their representational power. To address the above problem, Chen et al. [116] proposed *dynamic convolution*, a novel operator design that increases representational power with negligible additional computational cost and does not change the width or depth of the network in parallel with CondConv [173].

Dynamic convolution uses K parallel convolution kernels of the same size and input/output dimensions instead of one kernel per layer. Like SE blocks, it adopts a squeeze-and-excitation mechanism to generate the attention weights for the different convolution kernels. These kernels are then aggregated dynamically by weighted summation and applied to the input feature map X:

$$s = \mathrm{softmax}(W_2 \delta(W_1 \mathrm{GAP}(X))) \tag{75}$$

$$\mathrm{DyConv} = \sum_{i=1}^{K} s_k \mathrm{Conv}_k \tag{76}$$

$$Y = \mathrm{DyConv}(X) \tag{77}$$

Here the convolutions are combined by summation of weights and biases of convolutional kernels.

Compared to applying convolution to the feature map, the computational cost of squeeze-and-excitation and weighted summation is extremely low. Dynamic convolution thus provides an efficient operation to improve representational power and can be easily used as a replacement for any convolution.

3.6 Channel & Spatial Attention

Channel & spatial attention combines the advantages of channel attention and spatial attention. It adaptively selects both important objects and regions [50]. The *residual attention network* [119] pioneered the field of channel & spatial attention, emphasizing the importance of informative features in both spatial and channel dimensions. It adopts a bottom-up structure consisting of several convolutions to produce a 3D (height, width, channel) attention map. However, it has high computational cost and limited receptive fields.

To leverage global spatial information later works [6], [117] enhance discrimination of features by introducing global average pooling, as well as decoupling channel attention and spatial channel attention for computational